June 26, 2026

Data Transformation Manipulation & Exploration in SQL

A structured way to turn dirty data into decisions that move.

Facilitators: Martha Klenam Tsagli & Eric Junior Nyarko-Sampson

Today's Agenda

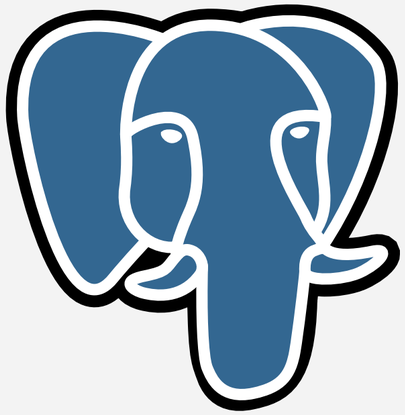
1. Exploring Data & Data Types
2. Lab 1 – Profile the Orders Table
3. Joins & Core Functions
4. Lab 2 – Distributor joins & name cleaning
5. Aggregation + Dates & Text
6. CEO Report (Final Lab)

Learning Objectives

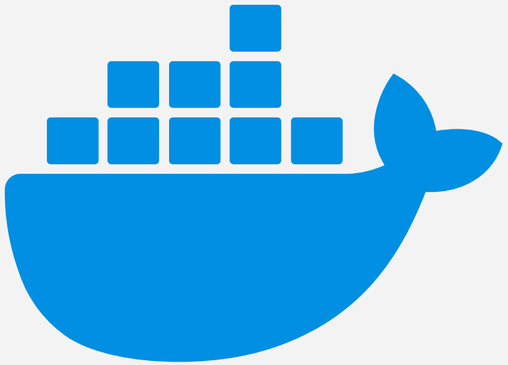
1. Profile a raw table for nulls, duplicates, and type issues.
2. Join multiple tables while preserving unmatched rows.
3. Clean and standardise inconsistent text data.
4. Aggregate sales data by region and time period.
5. Structure a multi-step transformation pipeline using CTEs (Common Table Expressions)

Setup Check

OPTION A: pgAdmin 4



OPTION B: Docker Installation



Icebreaker

The Scenario

For the rest of this session, this is your **reality**.

You've just joined **Indomie Ghana's** data team as a **Data Analyst**

Today is your first real assignment. Abena has dropped **four raw database tables** on your desk, **orders**, **distributors**, **product variants**, and **delivery dates**, and asked you to make sense of them.

Nobody has cleaned these tables. Nobody has validated them. There are missing values, inconsistent region names, suspiciously large order quantities, and duplicate rows baked in. Your job is to **profile, clean,** and **summarise** this data into something Abena can actually use.

Exploring Data & Datatypes

Abena's brief

"Before you touch anything, tell me what we are working with? Are there any obvious issues?"

What tools do we reach for?

When Abena asks "what are we working with?", SQL gives us some answers.

COUNT(*) / COUNT(col)

How many rows do we have, and how many are missing data?

GROUP BY

How does the data break down by region, status, or distributor?

HAVING

Which groups meet a condition, e.g. appear more than once?

CASTING

Are our columns the right data type to work with?

What tools do we reach for?

When Abena asks "what are we working with?", SQL gives us some answers.

COUNT(*) / COUNT(col)

COUNT(*) counts every row, including those with NULL values.

COUNT(column_name) counts only the rows where that column is not NULL. This matters when checking for missing data.

What tools do we reach for?

When Abena asks "what are we working with?", SQL gives us some answers.

COUNT(*) / COUNT(col)

```
SELECT COUNT(*) FROM distributors;

SELECT COUNT(*) FROM orders;

SELECT
COUNT(*) AS total_rows,
COUNT(distributor_id) AS non_null_distributor_id,
COUNT(variant_id) AS non_null_variant_id,
COUNT(order_date) AS non_null_order_date,
COUNT(region) AS non_null_region,
FROM orders;
```

What tools do we reach for?

When Abena asks "what are we working with?", SQL gives us some answers.

COUNT(*) / COUNT(col)

What is this query doing?

```
SELECT
  COUNT(*) AS total_rows,
  COUNT(quantity_cartons) AS rows_with_quantity,
  COUNT(*) - COUNT(quantity_cartons) AS missing_quantity
FROM orders;
```

What tools do we reach for?

When Abena asks "what are we working with?", SQL gives us some answers.

GROUP BY

GROUP BY collapses rows that share the same value in a column into a single summary row. You must use an aggregate function (like COUNT, SUM, AVG) on every other column you SELECT.

What tools do we reach for?

When Abena asks "what are we working with?", SQL gives us some answers.

GROUP BY

```
SELECT distributor_name AS cleaned_name,
COUNT(*) AS occurrences
FROM distributors
GROUP BY distributor_name
HAVING COUNT(*) > 1
ORDER BY occurrences DESC;
```

What tools do we reach for?

When Abena asks "what are we working with?", SQL gives us some answers.

HAVING

WHERE filters rows before aggregation.

HAVING filters groups after aggregation. Use HAVING when your condition involves an aggregate like COUNT or SUM.

What tools do we reach for?

When Abena asks "what are we working with?", SQL gives us some answers.

HAVING

```
SELECT
  distributor_id,
  variant_id,
  order_date,
  COUNT(*) AS occurrences
FROM orders
GROUP BY distributor_id, variant_id, order_date
HAVING COUNT(*) > 1;
```

What tools do we reach for?

When Abena asks "what are we working with?", SQL gives us some answers.

CASTING

Casting tells the database to treat a value as a different data type.
PostgreSQL supports two syntaxes, both do the same thing.

What tools do we reach for?

When Abena asks "what are we working with?", SQL gives us some answers.

CASTING

```
SELECT order_id, CAST(order_date AS VARCHAR) AS order_date_text FROM orders;
```

```
SELECT
  order_id,
  actual_date,
  expected_date,
  CAST(actual_date - expected_date AS INT) AS days_to_deliver
FROM delivery_dates;
```

Lab 1 – Profile the Orders Table

1. How many orders do we have?
2. Which order status appears the most?
3. Which region placed the most orders?
4. Are there any missing quantities in the orders table?
5. Which regions had more than 10 orders?

Joins and Core Functions

Abena's brief

I need distributor names attached to every order

What tools do we reach for?

When Abena asks "I need distributor names attached to an order?", What do we do?

JOINS

Combine rows from two tables based on a matching column.

UPPER/LOWER

Normalize text casing so "West" and "WEST" don't get treated as different values.

TRIM

Strip leading and trailing whitespace so "Acme " and "Acme" match correctly.

COALESCE

Return the first non-NULL value from a list, useful for filling in fallback values

What tools do we reach for?

When Abena asks "I need distributor names attached to an order?", What do we do?

JOINS

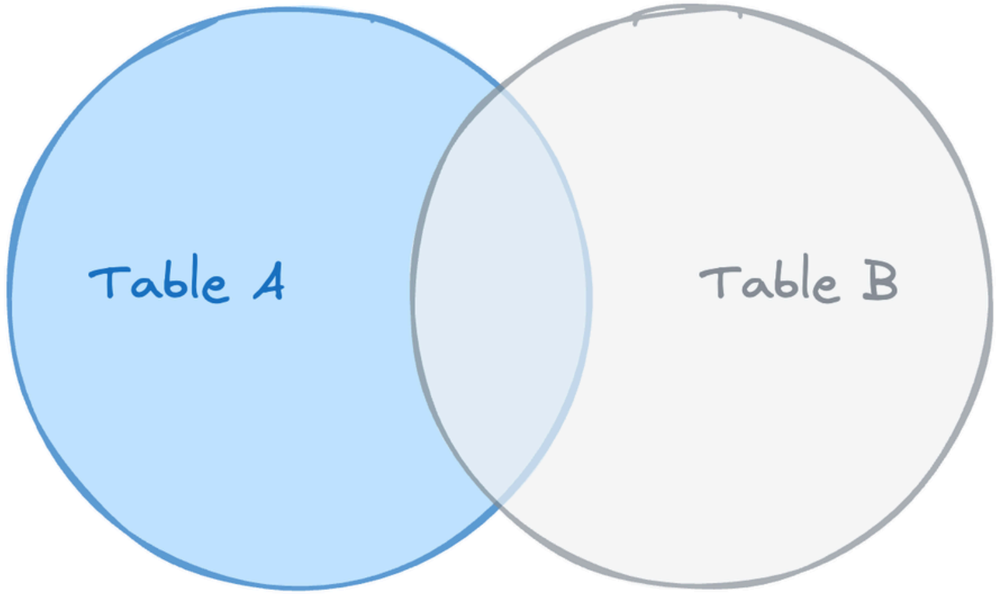
JOIN combines rows from two or more tables based on a related column. The type of join determines which rows are kept.

INNER JOIN; returns only rows where a match exists in both tables.

LEFT JOIN; returns all rows from the left table and matching rows from the right. Rows with no match on the right show NULL.

RIGHT JOIN; returns all rows from the right table, and only the matching rows from the left table. Rows with no match on the left will show NULL.

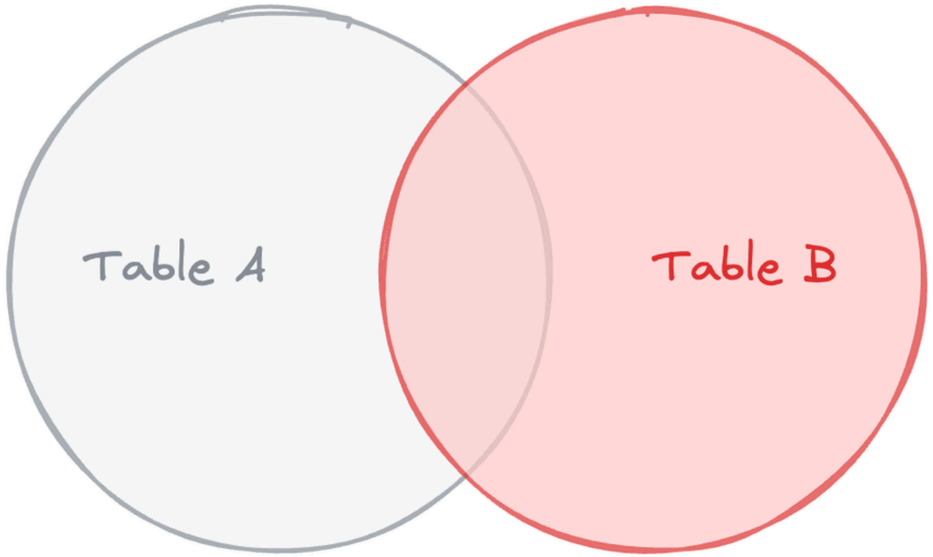
LEFT JOIN



Returns *ALL* rows from Table *A* (left),
and matched rows from Table B. Unmatched B rows = *NULL*.

```
SELECT * FROM A LEFT JOIN B ON A.id = B.id
```

RIGHT JOIN



Returns ALL rows from Table B (right),
and matched rows from Table A. Unmatched A rows = NULL.

```
SELECT * FROM A RIGHT JOIN B ON A.id = B.id
```

What tools do we reach for?

When Abena asks "I need distributor names attached to an order?", What do we do?

JOINS

```
SELECT
  d.distributor_id,
  d.distributor_name,
  COUNT(o.order_id) AS total_orders
FROM distributors d
LEFT JOIN orders o ON d.distributor_id = o.distributor_id
GROUP BY d.distributor_id, d.distributor_name
ORDER BY total_orders ASC
```

What tools do we reach for?

When Abena asks "I need distributor names attached to an order?", What do we do?

LOWER, UPPER & TRIM

Text cleaning functions help standardise inconsistent string data before analysis.

LOWER(text) - converts all characters to lowercase: 'Ashanti' → 'ashanti'.

UPPER(text) — converts all characters to uppercase.

TRIM(text) — removes leading and trailing spaces. Use when data entry leaves invisible whitespace.

These are often chained together: **LOWER(TRIM(column))**

What tools do we reach for?

When Abena asks "I need distributor names attached to an order?", What do we do?

LOWER, UPPER & TRIM

```
-- Standardise region and distributor name for comparison
SELECT
  LOWER(TRIM(region)) AS clean_region,
  UPPER(TRIM(distributor_name)) AS clean_name
FROM distributors
ORDER BY clean_region;
```

```
-- Count orders per normalised region
SELECT
  LOWER(TRIM(o.region)) AS region,
  COUNT(*) AS order_count,
  SUM(o.quantity_cartons) AS total_cartons
FROM orders o
WHERE o.quantity_cartons IS NOT NULL
GROUP BY LOWER(TRIM(o.region))
ORDER BY total_cartons DESC;
```

What tools do we reach for?

When Abena asks "I need distributor names attached to an order?", What do we do?

COALESCE

COALESCE(value1, value2, ...) returns the first non-NULL value from a list of arguments.

Use it to replace NULLs with a default value in your result set so downstream calculations don't break.

What tools do we reach for?

When Abena asks "I need distributor names attached to an order?", What do we do?

COALESCE

```
SELECT order_id, COALESCE(quantity_cartons, 0) AS quantity_cartons
FROM orders
WHERE quantity_cartons IS NULL;

SELECT SUM(COALESCE(quantity_cartons, 0)) AS total_cartons FROM orders;
```

Lab 2 – Distributor Joins & Name Cleaning

1. Which distributors have never placed an order?
2. Clean up the region column?
3. Which distributor ordered the most cartons?
4. Spot similar distribution names.

Aggregation + Dates & Text

Abena's brief

I need to see monthly sales totals by variant, average delivery delays, and total carton value per region. Structure it so I can drop it straight into the board deck.

What tools do we reach for?

When Abena asks "let’s get monthly sales and other related data", What do we do?

SUM, AVG, MIN, MAX

SUM; adds everything up

AVG; divides the total by the count:

MIN / MAX; scans all rows and picks the smallest or largest:

What tools do we reach for?

When Abena asks "let's get monthly sales and other related data", What do we do?

SUM, AVG, MIN, MAX

```
SELECT SUM(quantity_cartons) AS total_cartons FROM orders;
```

```
SELECT AVG(quantity_cartons) AS average_cartons FROM orders;
```

```
SELECT
MIN(quantity_cartons) AS smallest_order,
MAX(quantity_cartons) AS largest_order
FROM orders;
```

```
SELECT SUM(quantity_cartons) AS
total_cartons,
    AVG(quantity_cartons) AS
average_cartons,
    MIN(quantity_cartons) AS
smallest_order,
    MAX(quantity_cartons) AS
largest_order
FROM orders;
```

What tools do we reach for?

When Abena asks "let’s get monthly sales and other related data", What do we do?

DATE_TRUNC & Date Arithmetic

DATE_TRUNC('period', date) truncates a date to the start of the specified time period.

DATE_TRUNC('month', order_date) first day of the order's month. Useful for grouping by month.

What tools do we reach for?

When Abena asks "let's get monthly sales and other related data", What do we do?

DATE_TRUNC & Date Arithmetic

```
SELECT
  AVG(actual_date - expected_date) AS avg_delay_days,
  MAX(actual_date - expected_date) AS max_delay_days
FROM delivery_dates
WHERE delivery_status = 'delayed';

SELECT
  AVG(actual_date - expected_date) AS avg_delay_days,
  MAX(actual_date - expected_date) AS max_delay_days
FROM delivery_dates
WHERE delivery_status = 'delayed';
```

What tools do we reach for?

When Abena asks "let’s get monthly sales and other related data", What do we do?

Common Table Expressions (CTEs)

A CTE (defined with WITH) is a named temporary result set you can reference in the main query.

Why use CTEs?
They break complex transformations into readable, step-by-step stages.

Each CTE can reference earlier ones, building a pipeline.
They make it easy to inspect intermediate results during development.